Selection Sort — Analysis and Reflection

Selection Sort divides the array into a sorted and an unsorted region. On each pass it scans the unsorted region to find the minimum element and swaps it into the next sorted position. It runs in O(n2) time in all cases — best, average, and worst — and uses O(1) auxiliary space, making it in-place. Unlike Insertion Sort it is not stable by default, as swaps can change the relative order of equal elements.

Best Case. On an already-sorted array of 500 elements the measured time was 4.96 ms. Unlike Insertion Sort, Selection Sort gains no benefit from sorted input because it must still scan the entire unsorted region on every pass to find the minimum, regardless of the current order. Best and worst case performance are essentially identical.

Worst Case. A reversed 500-element array took 4.99 ms, virtually the same as the best case. This illustrates a key characteristic of Selection Sort: its O(n2) comparison count is fixed at n(n-1)/2 no matter the input, because the inner loop always runs to completion.

Average Case. Random arrays averaged 0.17 ms at n=100, 0.60 ms at n=200, 1.49 ms at n=300, 3.19 ms at n=400, and 5.12 ms at n=500. Growth is clearly quadratic, consistent with the theoretical O(n2) average case. The constant is comparable to Insertion Sort on random data but without any best-case advantage.

Space Complexity. Selection Sort is in-place, requiring only a single temporary variable for the swap regardless of input size — O(1) auxiliary space. This makes it suitable for memory-constrained environments, though its lack of best-case advantage limits its practical appeal compared to Insertion Sort.

Stability. Sorting [(3,alice),(1,bob),(2,carol),(1,dave),(2,eve)] by the first element yields [(1,bob),(1,dave),(2,carol),(2,eve),(3,alice)]. In this run the relative order of equal keys happened to be preserved, but standard Selection Sort is not guaranteed to be stable. A swap can move an element past others with the same key, so stability depends on implementation and input. A stable variant can be achieved by inserting rather than swapping, at the cost of extra shifts.

Efficiency Discussion. Selection Sort performs the same number of comparisons regardless of input order, making it predictable but rarely optimal. It makes at most n-1 swaps, which is an advantage when swaps are expensive relative to comparisons. For general use Insertion Sort is preferable on small or nearly-sorted data due to its O(n) best case, while Quick Sort and Merge Sort are far better for large datasets. Bubble Sort is similar in complexity but performs more swaps and offers no advantage over either alternative.

Practical Applications. Selection Sort is useful when the cost of writing to memory is high and the number of swaps must be minimised, such as on flash storage with limited write cycles. It is also a straightforward teaching algorithm for introducing the concept of in-place sorting. Beyond these niche cases its consistent O(n2) behaviour makes it less practical than Insertion Sort for nearly-sorted data or than O(n log n) algorithms for large datasets.

Improvements and Variations. Heap Sort can be seen as an optimised Selection Sort that uses a max-heap to find the minimum in O(log n) rather than O(n), bringing overall complexity down to O(n log n) while remaining in-place. Bidirectional Selection Sort (sometimes called Cocktail Selection Sort) finds both the minimum and maximum on each pass, roughly halving the number of passes though the asymptotic complexity remains O(n2). For stability, a linked-list implementation allows insertion instead of swapping, preserving relative order of equal keys without extra space overhead.

Unit Tests. Three regular cases: [3,1,2] sorts to [1,2,3]; [1,2,3] remains [1,2,3]; [-3,-1,-2] sorts to [-3,-2,-1]. Three edge cases: an empty list returns an empty list; a single-element list [42] is unchanged; duplicates [2,2,1,1] sort correctly to [1,1,2,2]. All six tests pass.